

CODEC-CORTEX: Roadmap de Recomendaciones por Fases

Volumen 2 — Informe de Recomendaciones. Roadmap por fases (0.2.0 → 0.3.0 → 1.0.0), dependencias entre fases, métricas de éxito por versión, riesgos del roadmap y acciones inmediatas ejecutables esta semana. Tono técnico-directo.

Agente autónomo · Protocolo PROTOCOLO_BENCHMARK v0.1.0

SKILL.cortex v0.2.3 (specification/alpha) · canon único de entrada
Complemento del Volumen 1 — Reporte Completo de Benchmark.

Tabla de contenidos

Capítulo 1 — Resumen ejecutivo	3
1.1 Roadmap en una tabla	3
1.2 Mensaje clave	3
Capítulo 2 — Hallazgos críticos del benchmark	4
Capítulo 3 — Roadmap corto plazo (v0.2.0, ~4 semanas)	6
3.1 Objetivos de v0.2.0	6
O-020-1: Implementar TPER (M-CAND-001)	6
O-020-2: Implementar BOM (M-CAND-004)	6
O-020-3: Corregir BVR de head_tail_raw	7
O-020-4: Documentar limitación UC_FPR/CFCR en README	7
O-020-5: Añadir tests de regresión para invariantes de CPP	8
3.2 Hitos de v0.2.0	8
Capítulo 4 — Roadmap medio plazo (v0.3.0, ~8–12 semanas)	9
4.1 Objetivos de v0.3.0	9
O-030-1: Subir corpus a Level 1 (8–12 casos, 2+ dominios)	9
O-030-2: Implementar SA-BC-FNR (M-CAND-002)	9
O-030-3: Implementar SCR (M-CAND-003)	10
O-030-4: Validar TPER en Level 1 y promover a validated	10
O-030-5: Añadir 2 variantes cortex declaradas planned (adaptive, semantic_hybrid)	11
O-030-6: Re-ejecutar benchmark con 2+ tokenizers reales	11
4.2 Hitos de v0.3.0	12
Capítulo 5 — Roadmap largo plazo (v1.0.0, ~6 meses)	13
5.1 Objetivos de v1.0.0	13

O-100-1: Implementar codec determinístico (parser, AST, encode/decode, verify)	13
O-100-2: Ejecutar fase LLM separada (§11 del protocolo)	13
O-100-3: Subir corpus a Level 2/3 (multidominio + adversarial)	14
O-100-4: Runtime lifecycle (compress / promote / decay)	14
O-100-5: Enterprise MCP design	15
5.2 Hitos de v1.0.0	16
Capítulo 6 — Dependencias entre fases	17
6.1 Grafo de dependencias	17
6.2 Camino crítico	17
6.3 Consecuencias de retrasar una fase	17
Capítulo 7 — Métricas de éxito por fase	18
7.1 Métricas de éxito v0.2.0	18
7.2 Métricas de éxito v0.3.0	18
7.3 Métricas de éxito v1.0.0	18
Capítulo 8 — Riesgos del roadmap	20
Capítulo 9 — Acciones inmediatas (esta semana)	21

Capítulo 1 — Resumen ejecutivo

Este informe de recomendaciones complementa el Volumen 1 (Reporte Completo de Benchmark) y traduce sus hallazgos en un **roadmap por fases** con tres horizontes claros: corto plazo (v0.2.0, ~4 semanas), medio plazo (v0.3.0, ~8–12 semanas) y largo plazo (v1.0.0, ~6 meses). El tono es técnico-directo: cada recomendación se enuncia como un imperativo con justificación inmediata, sin rodeos ni lenguaje hedged.

El mensaje central: **el benchmark actual valida la causalidad arquitectónica de cortex_priority_pack dentro del alcance Level 0, pero no permite claims de generalización**. El siguiente paso científico es desbloquear esas claims subiendo el corpus a Level 1; el siguiente paso ingenieril es cerrar los gaps métricos con TPER y SA-BC-FNR; el siguiente paso de madurez es implementar el codec determinístico que automatice las operaciones actualmente manuales.

1.1 Roadmap en una tabla

Versión	Horizonte	Hito principal	Claims desbloqueadas
v0.2.0	~4 semanas	TPER + BOM + tests de regresión	Refinar claim #8 (planned vs current)
v0.3.0	~8–12 semanas	Level 1 corpus + SA-BC-FNR + tokenizers reales	Claims de validez externa (generalización)
v1.0.0	~6 meses	Codec determinístico + CLI + fase LLM separada	Claims operacionales y de razonamiento LLM

Tabla 1.1 — Roadmap por fases. Cada versión desbloquea un tipo distinto de claim.

Las dependencias entre fases son estrictas: v0.3.0 requiere v0.2.0 (TPER debe estar validada antes de subir a Level 1); v1.0.0 requiere v0.3.0 (el codec determinístico requiere corpus Level 1+ para validar). El capítulo 6 detalla el grafo de dependencias y el camino crítico.

1.2 Mensaje clave

No publicar claims operacionales hasta v1.0.0. Cualquier claim sobre mejora de agentes LLM, calidad de respuesta o outcomes operacionales requiere fase LLM separada (§11 del protocolo). Hasta entonces, todas las comunicaciones externas deben limitarse a: *preservación de evidencia bajo compresión contextual, validada dentro del corpus probado*. Esta restricción no es opcional: es la regla §1.4 de no sobreafirmación.

Capítulo 2 — Hallazgos críticos del benchmark

Diez hallazgos priorizados por urgencia de acción. Cada hallazgo se acompaña de su evidencia (referencia al Volumen 1) y su implicación para el roadmap.

#	Hallazgo	Evidencia (Vol.1)	Implicación	Urgencia
1	CPP preserva P0 entries en todos los presupuestos 256–8192 tokens.	Vol.1 §9.2 Tabla 9.2. P0_Survival=1.000 en 11 presupuestos × 3 escenarios.	Es la propiedad más robusta del benchmark. Requiere tests de regresión que la protejan (acción inmediata #5).	ALTA — proteger
2	CPP preserva blocking constraints (BC_FNR=0.000).	Vol.1 §9.5 Tabla 9.5. Confirmado con 3 blocking CNST en caso agent.	Es la conclusión operacional más fuerte. Requiere tests de regresión específicos para cada blocking CNST del corpus.	ALTA — proteger
3	Causalidad arquitectónica confirmada: P0 + CNST son mecanismos necesarios.	Vol.1 §10.1 Tabla 10.1. ablation_no_P0 y ablation_no_constraints colapsan a BC_FNR=0.676.	Documentar esta causalidad en SKILL.cortex como nota normativa. Cualquier cambio futuro a CPP debe preservar P0 immutability y CNST preservation.	ALTA — documentar
4	CFCR/UC_FPR sobre-cuentan porque flaggean presencia, no mala representación.	Vol.1 §9.4 y §9.6. Todos los métodos muestran CFCR ~0.18 incluso cuando preservan status:planned correctamente.	Implementar TPER (M-CAND-001) en v0.2.0 como prioridad máxima. Sin TPER, claim #8 queda estancada en Partially supported.	ALTA — resolver (v0.2.0)
5	BC_FNR no detecta pérdida de severity (ablation_no_severity gap).	Vol.1 §10.2. ablation_no_severity mantiene BC_FNR=0.000 pero elimina el atributo severity=blocking.	Implementar SA-BC-FNR (M-CAND-002) en v0.3.0. Cierra el gap entre 'entrada preservada' y 'severidad preservada'.	MEDIA — resolver (v0.3.0)
6	BVR binaria oculta magnitud de violación.	Vol.1 §13.1 Patrón 4. head_tail_raw muestra BVR=0.182 por 2 tokens de overflow del conector '... '.	Implementar BOM (M-CAND-004) en v0.2.0. Corregir head_tail_raw para que respete budget exacto.	BAJA — refinar (v0.2.0)
7	Corpus Level 0 no permite claims de generalización.	Vol.1 §14.2. Solo 3 casos internos; §4.3 del protocolo lo prohíbe explícitamente.	Subir a Level 1 (8–12 casos, 2+ dominios) en v0.3.0. Es el desbloqueador de claims de validez externa.	ALTA — desbloquear (v0.3.0)
8	Tokenizer proxy whitespace_split limita transferencia de umbrales absolutos.	Vol.1 §14.1. Los conteos absolutos no coinciden con BPE/WordPiece/SentencePiece.	Re-ejecutar benchmark con 2+ tokenizers reales en v0.3.0. Validar que el ordenamiento relativo de métodos se mantiene.	MEDIA — validar (v0.3.0)
9	No se ejecutó fase LLM; claims operacionales prohibidas.	Vol.1 §16.3 N1. Protocolo §1.4 y §11 lo exigen.	Ejecutar fase LLM en v1.0.0 con 3 modelos × 3 prompts × 5 repeticiones. Hasta entonces, no publicar claims de mejora LLM.	ALTA — desbloquear (v1.0.0)
10	Codec determinístico, CLI, runtime y MCP están planned/future, no implementados.	Vol.1 §2.2 Tabla 2.1. SKILL.cortex declara states explícitamente.	Implementar codec + CLI en v1.0.0. Hasta entonces, comunicar externamente que CODEC-CORTEX es Skill + .cortex + HCORTEX manual.	ALTA — desbloquear (v1.0.0)

Tabla 2.1 — Diez hallazgos críticos priorizados por urgencia de acción.

Los hallazgos 1–3 son propiedades a **proteger** (regresiones prohibidas); los hallazgos 4–6 son **gaps a resolver** con métricas candidatas; los hallazgos 7–8 son **limitaciones a desbloquear** subiendo corpus y tokenizers; los hallazgos 9–10 son **capacidades a implementar** para desbloquear claims operacionales. Las tres fases del roadmap atacan estos cuatro grupos en orden.

Capítulo 3 — Roadmap corto plazo (v0.2.0, ~4 semanas)

La versión 0.2.0 cierra los gaps métricos más urgentes identificados en el Volumen 1. Es una versión de **consolidación**: no añade corpus ni métodos, sólo refina las métricas existentes y protege las propiedades validadas con tests de regresión.

3.1 Objetivos de v0.2.0

Cinco objetivos, cada uno con scope, deliverables, dependencies, success criteria y risk.

O-020-1: Implementar TPER (M-CAND-001)

Motivation: CFCR y UC_FPR sobre-cuentan falsos positivos (Vol.1 §9.4, §13.2). Sin TPER, claim #8 'CPP distingue planned de current' queda estancada en Partially supported.

Scope: Implementar métrica TPER en metrics.py. Para cada CLAIM/STAT/HDL con status en {planned, future, blocked}, verificar si el contexto presenta el claim como current (sin calificador status en N tokens). $TPER = \text{misinterpreted} / \text{total_non_current_entries}$.

Deliverables:

- metrics.py: metric_TPER()
- metric_registry.json: M-CAND-001 promoted to experimental
- tests/test_tper.py: al menos 5 casos de prueba

Dependencies: ['Ninguna; es aditiva']

Success criteria:

- TPER implementada y reproducible
- TPER distingue CPP (TPER=0) de ablation_no_temporal (TPER>0) en stale_state_conflict
- Claim #8 promovible a Supported

Risk: BAJO. Definición de 'N tokens' puede requerir tuning empírico; empezar con N=50.

O-020-2: Implementar BOM (M-CAND-004)

Motivation: BVR binaria trata 2 tokens de overflow igual que 2000 tokens (Vol.1 §13.1 Patrón 4).

Scope: Implementar $BOM = \max(0, \text{context_tokens} - \text{budget}) / \text{budget}$ en metrics.py.

Deliverables:

- metrics.py: metric_BOM()
- metric_registry.json: M-CAND-004 promoted to experimental

Dependencies: ['Ninguna']

Success criteria:

-
- BOM implementada
 - BOM reporta magnitud real de overflow para head_tail_raw (~0.005) vs violaciones graves (>0.1)

Risk: BAJO.

O-020-3: Corregir BVR de head_tail_raw

Motivation: head_tail_raw muestra BVR=0.182 por el conector ' ... ' que añade 2 tokens (Vol.1 §13.1 Patrón 4). Es un defecto de implementación, no un hallazgo científico.

Scope: Modificar m_head_tail_raw en methods.py para que compute budget después del conector.

Deliverables:

- methods.py: fix en m_head_tail_raw
- tests/test_head_tail_budget.py

Dependencies: ['Ninguna']

Success criteria:

- BVR=0.000 para head_tail_raw en todos los presupuestos

Risk: BAJO.

O-020-4: Documentar limitación UC_FPR/CFCR en README

Motivation: Hasta que TPER esté validada, los usuarios del benchmark deben saber que UC_FPR/CFCR flaggean presencia, no mala representación.

Scope: Añadir sección 'Known limitations' en README.md y en metric_specs/UC_FPR.json, CFCR.json.

Deliverables:

- README.md: nueva sección
- metric_specs/UC_FPR.json: campo limitations
- metric_specs/CFCR.json: campo limitations

Dependencies: ['Ninguna']

Success criteria:

- Limitación documentada públicamente
- Referencia cruzada a TPER como métrica sucesora

Risk: NINGUNO.

O-020-5: Añadir tests de regresión para invariantes de CPP

Motivation: P0_Survival=1.000 y BC_FNR=0.000 son las propiedades más robustas del benchmark (Vol.1 hallazgos 1 y 2). Cualquier cambio futuro que las rompa debe detectarse inmediatamente.

Scope: Crear tests/test_cpp_invariants.py que ejecute CPP en blocking_constraint_survival y reduced_window_compression y afirme P0_Survival=1.000 y BC_FNR=0.000.

Deliverables:

- tests/test_cpp_invariants.py
- CI hook para correr en cada PR

Dependencies: ['Ninguna']

Success criteria:

- Tests pasan en CI
- Tests fallan si se aplica ablation_no_P0 a CPP (verificación negativa)

Risk: BAJO.

3.2 Hitos de v0.2.0

Hito	Semana	Criterio de salida
Branch v0.2.0 creada	Semana 1	branch creada, manifest bump a 0.2.0-dev
TPER implementada y tested	Semana 2	tests/test_tper.py pasa; metric_registry actualizado
BOM implementada + head_tail_raw fix	Semana 2	BVR=0.000 para head_tail_raw en todos los presupuestos
Tests de regresión CPP en CI	Semana 3	tests/test_cpp_invariants.py pasa en CI
README + metric_specs actualizados	Semana 3	limitaciones UC_FPR/CFCR documentadas
Re-ejecutar benchmark completo	Semana 4	3690+ filas scored, 0 errores, TPER y BOM en CSVs
Release v0.2.0	Semana 4	tag git, publish, claim #8 promovida a Supported

Tabla 3.1 — Hitos de v0.2.0 con criterios de salida verificables.

Capítulo 4 — Roadmap medio plazo (v0.3.0, ~8–12 semanas)

La versión 0.3.0 desbloquea claims de validez externa subiendo el corpus a Level 1, cierra el gap de severity con SA-BC-FNR, valida TPER en el nuevo corpus y añade tokenizers reales para validar la robustez del tokenizer proxy.

4.1 Objetivos de v0.3.0

O-030-1: Subir corpus a Level 1 (8–12 casos, 2+ dominios)

Motivation: Level 0 (3 casos) no permite claims de generalización (Vol.1 §14.2). Level 1 desbloquea claims internas moderadas.

Scope: Añadir 5–9 casos nuevos cubriendo al menos 2 dominios distintos (e.g., customer-support, code-review, content-generation). Cada caso en 5 representaciones. Hashes SHA-256 registrados.

Deliverables:

- corpus/source/ con 8–12 casos
- corpus/normalized/hashes.json actualizado
- tasks/tasks.jsonl expandido a 60+ tareas
- gold_answers.jsonl y gold_decisions.jsonl expandidos

Dependencies: ['v0.2.0 completado (TPER disponible para evaluar nuevos casos)']

Success criteria:

- ≥ 8 casos, ≥ 2 dominios
- Hashes verificables
- Cobertura 4 tipos $\times$ 8 escenarios mantenida
- Ningún caso sin gold answer

Risk: MEDIO. Conseguir casos reales anonimizados puede requerir acuerdos; alternatively, generar casos sintéticos realistas con domain experts.

O-030-2: Implementar SA-BC-FNR (M-CAND-002)

Motivation: BC_FNR no detecta pérdida de severity (Vol.1 §10.2, ablation_no_severity gap).

Scope: Para cada blocking CNST en gold, verificar (a) entry preservado Y (b) severity=blocking attribute preservado. SA-BC-FNR = missing_or_degraded / total_blocking.

Deliverables:

- metrics.py: metric_SA_BC_FNR()

-
- metric_registry.json: M-CAND-002 promoted to validated
 - tests/test_sa_bc_fnr.py

Dependencies: ['v0.2.0 completado']

Success criteria:

- SA-BC-FNR implementada
- SA-BC-FNR penaliza ablation_no_severity (debe ser > 0)
- SA-BC-FNR=0 para CPP en blocking_constraint_survival

Risk: BAJO.

O-030-3: Implementar SCR (M-CAND-003)

Motivation: STR binaria satura con una referencia a sigil (Vol.1 §13.1 Patrón 3).

Scope: SCR = unique_sigils_referenced_in_context / unique_sigils_in_source_case.

Deliverables:

- metrics.py: metric_SCR()
- metric_registry.json: M-CAND-003 promoted to experimental

Dependencies: ['Ninguna']

Success criteria:

- SCR implementada
- SCR distingue CPP (SCR alto) de head_json (SCR bajo) en multi_instance_sigil_preservation

Risk: BAJO.

O-030-4: Validar TPER en Level 1 y promover a validated

Motivation: TPER se implementa en v0.2.0 como experimental; v0.3.0 la valida en corpus más amplio.

Scope: Re-ejecutar benchmark con TPER sobre corpus Level 1. Verificar criterios §9.4: detecta fallos reales no visibles por métricas existentes, reproducible, no subjetiva, automáticamente computable, valor diagnóstico claro, no incentiva sobreajuste.

Deliverables:

- reports/tper_validation_report.md
- metric_registry.json: M-CAND-001 promoted to validated

Dependencies: ['O-030-1 (Level 1 corpus)', 'O-020-1 (TPER implementada)']

Success criteria:

-
- TPER detecta fallos en al menos 2 casos de Level 1
 - Reproducible en 2 runs
 - No depende de interpretación subjetiva

Risk: MEDIO. TPER podría no detectar fallos nuevos si Level 1 es demasiado similar a Level 0; diseñar casos con distorsión temporal deliberada.

O-030-5: Añadir 2 variantes cortex declaradas planned (adaptive, semantic_hybrid)

Motivation: v0.1.0 las declaró pero no implementó (Vol.1 §6.6). v0.3.0 las implementa para comparar.

Scope: Implementar `cortex_priority_pack_adaptive` (selección adaptativa de perfil según budget observado) y `cortex_priority_pack_semantic_hybrid` (CPP + perfiles semánticos).

Deliverables:

- `methods.py`: `m_cortex_adaptive()`, `m_cortex_semantic_hybrid()`
- `method_registry.json`: marcadas `implemented=true`
- `results` en `scored_tasks.csv`

Dependencies: ['v0.2.0 completado']

Success criteria:

- Ambas variantes implementadas y reproducibles
- Comparables con CPP v1 en todos los escenarios

Risk: MEDIO. Diseño de 'adaptive' y 'semantic_hybrid' puede requerir iteración.

O-030-6: Re-ejecutar benchmark con 2+ tokenizers reales

Motivation: Tokenizer proxy `whitespace_split` limita transferencia de umbrales absolutos (Vol.1 §14.1).

Scope: Añadir soporte para BPE (GPT-2 tokenizer) y WordPiece (BERT tokenizer) además de `whitespace_split`. Re-ejecutar benchmark con los tres tokenizers. Comparar ordenamiento relativo de métodos.

Deliverables:

- `scripts/tokenizers.py`: wrappers para BPE y WordPiece
- `runs/scored_tasks_bpe.csv`
- `runs/scored_tasks_wordpiece.csv`
- `reports/tokenizer_robustness_report.md`

Dependencies: ['v0.2.0 completado']

Success criteria:

- 3 tokenizers soportados
- Ordenamiento relativo de métodos se mantiene estable (>80% de agreement)
- Diferencias absolutas documentadas

Risk: MEDIO. Tokenizers reales pueden requerir dependencias externas (transformers, tokenizers).

4.2 Hitos de v0.3.0

Hito	Semana	Criterio de salida
Branch v0.3.0 creada	Semana 1	branch creada, manifest bump a 0.3.0-dev
Level 1 corpus diseñado	Semana 2–3	8–12 casos definidos con domain experts
Level 1 corpus implementado	Semana 4–5	todos los casos en 5 representaciones, hashes verificados
SA-BC-FNR + SCR implementadas	Semana 5	tests pasan, metric_registry actualizado
Variantes adaptive + semantic_hybrid	Semana 6–7	ambas implementadas y comparables
Tokenizers BPE + WordPiece integrados	Semana 8–9	3 tokenizers soportados
TPER validada en Level 1	Semana 10	validation_report emitido, M-CAND-001 promoted
Re-ejecutar benchmark completo	Semana 11	scored_tasks con 3 tokenizers, claims de validez externa desbloqueadas
Release v0.3.0	Semana 12	tag git, publish, claim #7 promotable a Supported

Tabla 4.1 — Hitos de v0.3.0 con criterios de salida verificables.

Capítulo 5 — Roadmap largo plazo (v1.0.0, ~6 meses)

La versión 1.0.0 es el hito de madurez: implementa el codec determinístico, publica la CLI, ejecuta la fase LLM separada y sube el corpus a Level 2/3. Es la versión que desbloquea claims operacionales y de razonamiento LLM.

5.1 Objetivos de v1.0.0

O-100-1: Implementar codec determinístico (parser, AST, encode/decode, verify)

Motivation: SKILL.cortex declara el codec como 'planned' (Vol.1 §2.2). Sin codec, las operaciones de parse/encode/decode son manuales o heurísticas.

Scope: Implementar parser .cortex con AST formal, encode/decode roundtrip, structural verify (validación de esquema), HCORTEX renderer automático. Cobertura de tests $\geq 90\%$.

Deliverables:

- codec/ module (Python)
- codec/parser.py, codec/ast.py, codec/encode.py, codec/decode.py, codec/verify.py, codec/renderer.py
- tests/codec/ con cobertura $\geq 90\%$
- CLI cortex publicada (cortex parse, cortex verify, cortex render)

Dependencies: ['v0.3.0 completado (corpus Level 1 sirve como fixture de tests)']

Success criteria:

- Roundtrip codec $\geq 99\%$ (parse→encode→parse produce mismo AST)
- CLI publicada en PyPI o como binario standalone
- Structural verify detecta entradas malformadas
- HCORTEX renderer produce salida conforme a §10 steps

Risk: ALTO. Es el objetivo más ambicioso del roadmap. Requiere diseño cuidadoso del AST y manejo de casos edge (entradas corruptas, atributos opcionales, backwards compatibility).

O-100-2: Ejecutar fase LLM separada (§11 del protocolo)

Motivation: Sin fase LLM, las claims operacionales están prohibidas (Vol.1 §16.3 N1). v1.0.0 desbloquea claims de calidad de respuesta y outcomes operacionales.

Scope: Mínimo 3 modelos (e.g., GPT-4, Claude, Gemini) $\times$ 3 plantillas de prompt $\times$ 5 repeticiones. Temperatura 0.0 como baseline; adicional 0.3 y 0.7. Evaluación de correctness, completeness y hallucination rate. Juez humano o LLM-as-judge con rúbrica fija.

Deliverables:

- llm_phase/ module

- llm_phase/prompts/ con 3 plantillas
- llm_phase/judge.py con rúbrica
- runs/llm_scored_tasks.csv
- reports/llm_phase_report.md separado del determinístico

Dependencies: ['v0.3.0 completado', 'presupuesto para API calls de LLM']

Success criteria:

- 3 modelos × 3 prompts × 5 reps ejecutados
- Reporte separado: Deterministic Evidence Preservation | Model-Based Response Quality | Operational Safety Outcome
- No mezclar resultados determinísticos y LLM en una sola métrica global (§11.2)

Risk: ALTO. Costo de API; variabilidad de LMs; diseño de rúbrica de judge.

O-100-3: Subir corpus a Level 2/3 (multidominio + adversarial)

Motivation: Level 1 permite claims internas moderadas; Level 2 (10 dominios, 2–3 casos por dominio) permite evaluar validez externa; Level 3 añade casos corruptos, conflictivos, ambiguos, incompletos y con evidencia enterrada.

Scope: Level 2: añadir casos hasta cubrir ≥ 10 dominios con 2–3 casos cada uno. Level 3: añadir casos adversariales (corruptos, conflictivos, ambiguos, incompletos, evidencia enterrada).

Deliverables:

- corpus/source/ expandido a Level 2 y Level 3
- corpus/normalized/hashes.json actualizado
- tasks/tasks.jsonl expandido a 200+ tareas
- reports/adversarial_robustness_report.md

Dependencies: ['v0.3.0 completado (Level 1)']

Success criteria:

- Level 2: ≥ 10 dominios, 2–3 casos por dominio
- Level 3: casos adversariales cubren 5 categorías (corruptos, conflictivos, ambiguos, incompletos, evidencia enterrada)
- CPP mantiene invariantes en casos adversariales o falla de forma segura

Risk: ALTO. Diseñar casos adversariales efectivos requiere ingeniería deliberada.

O-100-4: Runtime lifecycle (compress / promote / decay)

Motivation: SKILL.cortex declara runtime como 'future' (Vol.1 §2.2). Lifecycle automatiza consolidación de memoria.

Scope: Implementar runtime con política confirmada por usuario: compress (consolidar sesiones antiguas), promote (elevar entradas P1→P0 cuando son críticas), decay (reducir prioridad de entradas obsoletas). Política explícita y auditable.

Deliverables:

- runtime/ module
- runtime/policy.py con políticas declarativas
- tests/runtime/ con escenarios lifecycle
- reports/runtime_safety_report.md

Dependencies: ['O-100-1 (codec determinístico)']

Success criteria:

- Lifecycle ejecuta sin pérdida de P0 invariants
- Política es auditable (cada cambio se registra)
- Usuario puede revertir cualquier operación de runtime

Risk: ALTO. Políticas de memoria son difíciles de diseñar correctamente; reversibilidad es crítica.

O-100-5: Enterprise MCP design

Motivation: SKILL.cortex declara MCP como 'future' (Vol.1 §2.2). MCP expone operaciones con gobernanza y auditoría.

Scope: Diseñar (no necesariamente implementar en v1.0.0) la capa MCP: handlers para parse/encode/decode/verify/render/lifecycle, gobernanza (perms, audit), exposición como tools.

Deliverables:

- docs/mcp_design.md
- docs/mcp_handlers_spec.md
- proof-of-concept MCP server (opcional)

Dependencies: ['O-100-1 (codec)', 'O-100-4 (runtime)']

Success criteria:

- Design document completo
- Handlers especificados con input/output types
- Gobernanza y auditoría especificadas

Risk: MEDIO. Es diseño, no implementación; riesgo principal es scope creep.

5.2 Hitos de v1.0.0

Hito	Mes	Criterio de salida
Branch v1.0.0 creada	Mes 1	branch creada, manifest bump a 1.0.0-dev
Codec determinístico diseñado	Mes 1–2	AST spec, parser design, verify design
Codec implementado (parser, AST, encode/decode)	Mes 2–3	roundtrip $\geq 99\%$, cobertura tests $\geq 90\%$
CLI cortex publicada	Mes 3	cortex parse/verify/render funcionando
Fase LLM diseñada	Mes 3	3 prompts, 3 modelos, rúbrica de judge
Fase LLM ejecutada	Mes 4	3×3×5 reps completadas, reporte separado emitido
Level 2 corpus diseñado e implementado	Mes 4–5	≥ 10 dominios, 2–3 casos por dominio
Level 3 corpus adversarial	Mes 5	casos adversariales en 5 categorías
Runtime lifecycle implementado	Mes 5–6	compress/promote/decay con política auditable
MCP design document	Mes 6	docs/mcp_design.md completo
Release v1.0.0	Mes 6	tag git, publish, claims operacionales desbloqueadas

Tabla 5.1 — Hitos de v1.0.0 con criterios de salida verificables.

Capítulo 6 — Dependencias entre fases

Las tres fases no son paralelas; tienen dependencias estrictas. Este capítulo detalla el grafo de dependencias, el camino crítico y las consecuencias de retrasar una fase.

6.1 Grafo de dependencias

El grafo de dependencias entre los 16 objetivos (5 de v0.2.0 + 6 de v0.3.0 + 5 de v1.0.0) es el siguiente:

Origen	Destino	Tipo	Razón
O-020-1 (TPER)	O-030-4 (TPER validated)	blocking	TPER debe estar implementada antes de validarla en Level 1
O-030-1 (Level 1)	O-030-4 (TPER validated)	blocking	TPER se valida sobre corpus Level 1
O-030-1 (Level 1)	O-100-3 (Level 2/3)	blocking	Level 2 es extensión de Level 1
O-030-1 (Level 1)	O-100-1 (Codec)	blocking	Codec usa corpus Level 1 como fixture de tests
O-100-1 (Codec)	O-100-4 (Runtime)	blocking	Runtime requiere codec para parse/encode
O-100-1 (Codec)	O-100-5 (MCP design)	blocking	MCP expone operaciones del codec
O-100-4 (Runtime)	O-100-5 (MCP design)	blocking	MCP expone operaciones de runtime
O-030-1 (Level 1)	O-100-2 (Fase LLM)	soft	Fase LLM se beneficia de corpus más amplio
O-030-6 (Tokenizers)	O-100-2 (Fase LLM)	soft	Tokenizers reales mejoran fidelidad de fase LLM

Tabla 6.1 — Dependencias entre objetivos. 'blocking' = no se puede empezar sin el origen; 'soft' = recomendado pero no obligatorio.

6.2 Camino crítico

El camino crítico (cadena más larga de dependencias blocking) es:

O-020-1 (TPER) → O-030-1 (Level 1) → O-100-1 (Codec) → O-100-4 (Runtime) → O-100-5 (MCP design)

Este camino tiene una duración mínima de ~6 meses si no hay retrasos. Cualquier retraso en O-020-1 (TPER) o O-030-1 (Level 1) retrasa todo el roadmap. Por eso TPER es la prioridad máxima de v0.2.0 y Level 1 es la prioridad máxima de v0.3.0.

6.3 Consecuencias de retrasar una fase

Fase retrasada	Consecuencia	Mitigación
v0.2.0	TPER no disponible; claim #8 sigue Partially supported; v0.3.0 no puede empezar	Recursos dedicados a v0.2.0; no aceptar scope creep
v0.3.0	Level 1 no disponible; claims de generalización bloqueadas; v1.0.0 no puede empezar	Iniciar diseño de Level 1 en paralelo con v0.2.0 si hay recursos
v1.0.0	Codec/CLI/runtime/MCP siguen planned/future; claims operacionales prohibidas	Comunicar externamente que CODEC-CORTEX es Skill + .cortex + HCORTEX manual

Tabla 6.2 — Consecuencias y mitigaciones de retrasos por fase.

Capítulo 7 — Métricas de éxito por fase

Cada fase tiene umbrales cuantitativos de aceptación. Si una fase no los cumple, no se publica como release; se itera hasta cumplirlos.

7.1 Métricas de éxito v0.2.0

Métrica	Threshold	Verificación
TPER implementada y reproducible	Sí/No	tests/test_tper.py pasa en CI
TPER distingue CPP de ablation_no_temporal	TPER(CPP)=0, TPER(ablation)>0	Verificación en stale_state_conflict
BOM implementada	Sí/No	tests/test_bom.py pasa
BVR=0.000 para head_tail_raw en todos los presupuestos	0.000	tests/test_head_tail_budget.py
Tests de regresión CPP en CI	Sí/No	tests/test_cpp_invariants.py pasa; falla con ablation_no_P0
Claim #8 promovida a Supported	Sí/No	claim_matrix.md actualizada
Re-ejecución del benchmark sin errores	0 errores	errors.csv vacío

Tabla 7.1 — Métricas de éxito de v0.2.0. Si alguna falla, no publicar release.

7.2 Métricas de éxito v0.3.0

Métrica	Threshold	Verificación
Corpus Level 1 casos	≥8	corpus/source/ con ≥8 casos
Corpus Level 1 dominios	≥2	corpus_cases.json con ≥2 dominios
SA-BC-FNR implementada	Sí/No	tests/test_sa_bc_fnr.py pasa
SA-BC-FNR penaliza ablation_no_severity	SA-BC-FNR(ablation) >0	Verificación en blocking_constraint_survival
SCR implementada	Sí/No	tests/test_scr.py pasa
Variantes adaptive + semantic_hybrid implementadas	Sí/No	method_registry.json con implemented=true
Tokenizers reales soportados	≥2 (BPE, WordPiece)	scripts/tokenizers.py
Ordenamiento relativo de métodos estable	>80% agreement entre tokenizers	tokenizer_robustness_report.md
TPER validada en Level 1	Sí/No	tper_validation_report.md; M-CAND-001 promoted to validated
Claim #7 promotable a Supported	Sí/No	claim_matrix.md actualizada

Tabla 7.2 — Métricas de éxito de v0.3.0.

7.3 Métricas de éxito v1.0.0

Métrica	Threshold	Verificación
Codec roundtrip	≥99%	tests/codec/test_roundtrip.py

Métrica	Threshold	Verificación
Codec cobertura de tests	≥90%	coverage report
CLI cortex publicada	Sí/No	cortex parse/verify/render funcionando
Fase LLM modelos	≥3	llm_phase/ con 3 modelos
Fase LLM prompts	≥3	llm_phase/prompts/ con 3 plantillas
Fase LLM repeticiones	≥5 por (modelo, prompt)	llm_scored_tasks.csv
Reportes LLM separados de determinísticos	Sí/No	llm_phase_report.md separado
Level 2 dominios	≥10	corpus_cases.json con ≥10 dominios
Level 3 categorías adversariales	5 (corruptos, conflictivos, ambiguos, incompletos, enterrados)	adversarial_robustness_report.md
Runtime lifecycle implementado	Sí/No	runtime/ module con compress/promote/decay
Runtime política auditable	Sí/No	cada cambio registrado en audit log
MCP design document completo	Sí/No	docs/mcp_design.md
Claims operacionales desbloqueadas	Sí/No	claim_matrix.md con claims LLM Supported

Tabla 7.3 — Métricas de éxito de v1.0.0. Las más exigentes del roadmap.

Capítulo 8 — Riesgos del roadmap

Seis riesgos identificados, cada uno con probabilidad, impacto, mitigación y owner sugerido. Los riesgos se revisan al final de cada fase.

#	Riesgo	Probabilidad	Impacto	Mitigación	Owner
1	Overfitting al corpus Level 0	MEDIA	ALTO	Limitar v0.2.0 a 4 semanas; no añadir casos nuevos hasta v0.3.0; tests de regresión deben pasar también con fixtures sintéticas adicionales.	Tech lead del benchmark
2	Dependencia de tokenizer proxy	BAJA	MEDIO	v0.3.0 valida con 2+ tokenizers reales. Si el ordenamiento cambia, documentar como amenaza a la validez; no eliminar claims ya publicadas sin evidencia contradictoria.	Tech lead del benchmark
3	Scope creep en v1.0.0	ALTA	ALTO	Release incremental: v1.0.0 publica codec+CLI+fase LLM; v1.1.0 añade runtime; v1.2.0 añade MCP. Cada release tiene su propio scope frozen.	Product owner
4	Falta de fase LLM retrasa claims operacionales	MEDIA	MEDIO	Ejecutar fase LLM mínima en v0.4.0 (entre v0.3.0 y v1.0.0) con 1 modelo × 1 prompt × 3 repeticiones como proof-of-concept. No es definitiva pero da señales tempranas.	LLM team
5	Promoción prematura de métricas candidatas	MEDIA	ALTO	Checklist §9.4 obligatorio antes de cada promoción. Revisión por pares. Si una métrica se promueve y luego se descubre un problema, degradar a experimental y re-evaluar.	Tech lead del benchmark
6	Backwards compatibility con .cortex existentes	MEDIA	ALTO	Política de compatibility: cualquier .cortex válido en v0.x debe seguir siendo válido en v1.x. Tests de compatibilidad con corpus histórico. Migración voluntaria y progresiva (regla !progressive_compatibility de SKILL.cortex).	Codec team

Tabla 8.1 — Seis riesgos del roadmap con probabilidad, impacto, mitigación y owner.

Capítulo 9 — Acciones inmediatas (esta semana)

Siete acciones ejecutables esta semana, sin esperar a la planificación completa de v0.2.0. Cada acción tiene owner sugerido, deliverable y fecha objetivo.

#	Acción	Owner sugerido	Deliverable	Fecha objetivo
1	Crear branch v0.2.0	Tech lead	branch git v0.2.0-dev creada desde main	Lunes
2	Abrir issues para TPER (M-CAND-001) y BOM (M-CAND-004)	Tech lead	2 issues en GitHub con descripción, scope, deliverables, success criteria	Lunes
3	Corregir BVR de head_tail_raw	Cualquier ingeniero	PR con fix en methods.py + test test_head_tail_budget.py	Martes
4	Documentar limitación UC_FPR/CFCR en README	Tech lead	Sección 'Known limitations' en README.md + campos limitations en metric_specs/UC_FPR.json y CFCR.json	Miércoles
5	Añadir tests de regresión para invariantes de CPP	Cualquier ingeniero	tests/test_cpp_invariants.py con aserciones PO_Survival=1.000 y BC_FNR=0.000 para CPP en blocking_constraint_survival y reduced_window_compression	Jueves
6	Freeze de corpus Level 0 (versionado)	Tech lead	Tag git corpus-level-0-final + manifest.json con corpus_version=level-0-final	Jueves
7	Schedule de revisión para v0.3.0	Product owner	Meeting calendarizado para semana 4 de v0.2.0 con stakeholders; agenda: scope de Level 1, domain experts a invitar, presupuesto para casos sintéticos	Viernes

Tabla 9.1 — Siete acciones ejecutables esta semana. Sin dependencias entre ellas; pueden paralelizarse.

Mensaje de cierre. El benchmark actual es sólido dentro de su alcance; las seis claims Supported son defendibles y las cuatro Not supported están correctamente prohibidas. El roadmap v0.2.0 → v0.3.0 → v1.0.0 es la ruta para desbloquear las claims restantes sin sobreafirmar. La acción más urgente es implementar TPER; la acción más estratégica es subir a Level 1; la acción más ambiciosa es implementar el codec determinístico. Las siete acciones de esta semana preparan el terreno para las tres.